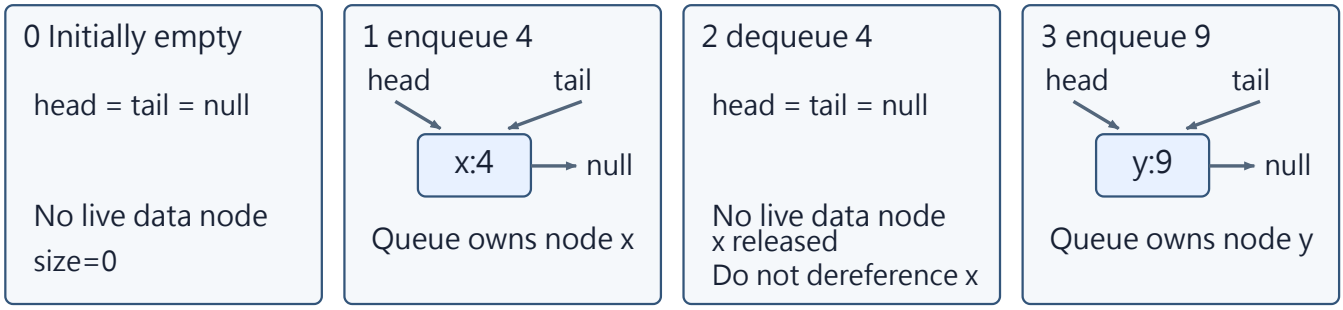
Chapter 4 worked solutions

DS04-E01 • The last queue node

Initially head=tail=null. Enqueue 4 creates x and sets head=tail=x and x.next=null. Dequeue saves its value, removes x, and sets head=null. Because the queue is empty, also set tail=null before releasing x. Enqueue 9 creates y and sets head=tail=y. If tail still points to x after deletion, a subsequent write to tail.next accesses a released object. This violates ownership and lifetime, not merely displayed contents.

Last-node removal: clear head and tail before release



DS04-E02 • Proving in-place reversal

Initially prev is empty and curr reaches $3 \to 8 \to 11$. After iteration 1, prev reaches 3 and curr $8 \to 11$. After iteration 2, prev reaches $8 \to 3$ and curr 11. After iteration 3, prev reaches $11 \to 8 \to 3$ and curr is empty. Saving curr.next before reversing the link preserves access to the remaining suffix. Both segments are acyclic, disjoint, and together contain exactly the original nodes. At termination set head=prev and tail=the old head. For an empty list both remain null. For a singleton they point to the same node.

DS04-E03 • Polynomial normalization and addition

P repeats exponent 4 and is not strictly decreasing. Combining terms gives $P=x^4+x$, while $Q=-x^4+5$. Their sum is $x+5$. The zero coefficient at degree four produces no node. Comparison-sorting p unordered input terms costs O(p log p), followed by O(p) combination. Only addition of two already canonical lists costs O(p+q). Unsorted-input normalization cannot be hidden in that merge bound.

DS04-E04 • Doubly linked deletion

Save p=x.prev and q=x.next, set p.next=q and q.prev=p, release x, and decrement size. All other links are unchanged. The modified neighbors satisfy p.next.prev=p and q.prev.next=q, restoring local consistency. The precondition is that x is a live data node owned by this container, is not a sentinel, and has not already been deleted. An interface accepting arbitrary external pointers needs ownership validation or a stronger handle contract. It cannot treat every pointer as valid.

This procedure fixes two nondata sentinels H and T. H.prev=null and T.next=null; an empty list has H.next=T, T.prev=H, and size=0. Every data node has nonnull prev/next, making p.next and q.prev valid. A null-ended representation instead needs explicit head/tail repair when erasing an endpoint and cannot directly use this procedure.

For one item H↔x↔T, p=H and q=T. Erasure restores H.next=T and T.prev=H and decreases size from 1 to 0. Never release H or T. The same sentinel links cover first/last data nodes in longer lists. Validate the live owned handle before entering pointer operations.

DS04-E05

Enqueue A gives `A→A`, tail=A; enqueue B gives `A→B→A`, tail=B. Removing A leaves `B→B`, tail=B. Before releasing B, set tail=null, leaving an empty queue. Enqueue C then gives `C→C`, tail=C. If tail still points to released B, reading `tail.next` on the next enqueue dereferences a dead object.

DS04-E06

$\mu = 2$ and $\lambda = 3$. The first-stage `(slow,fast)` pairs are `(1,2)`, `(2,4)`, `(3,3)`. Resetting one pointer to 0 gives `(0,3)`, `(1,4)`, `(2,2)`, so node 2 is the entry. Compare node identity or valid pointers, not stored values.

DS04-E07

On an empty chain fast is null and the first condition fails. On `A→null`, fast.next is null, so the loop does not run. On `A→A`, fast and fast.next are non-null and both pointers return to A on the first iteration. Short-circuit checks read fast.next only if fast is non-null, and read its next only if fast.next is non-null.

DS04-E08

After `p.next=x`, reading p.next gives x, so the second assignment makes `x.next=x`. Head reaches `3→5→5→⋯`; nodes 8 and 11 lose the owning path from head. An external address alone does not restore the container invariant. Save `old=p.next`, set `x.next=old`, then commit `p.next=x`, giving `3→5→8→11→null`.

DS04-E09

$\mu = 0$ and $\lambda = 2$. In round one slow=B and fast=A: both values are 7, but the nodes are distinct. In round two both pointers are A, which is the first meeting and entry. Resetting the other pointer to head=A already makes them equal. Comparing values would incorrectly report a meeting in round one.

DS04-E10

Save `p=x.prev=H` and `q=x.next=T`, set `p.next=q` and `q.prev=p`, then release x and decrement size from 1 to 0. The result has `H.next=T` and `T.prev=H`; both sentinels remain live. External pointers to released x are invalid to dereference.

DS04-E11

Shallow copy makes both heads refer to the same chain. If both destructors release it, the second release or access is invalid and causes undefined behavior. Deep copy allocates an independent chain with an explicit failure policy; a noncopyable class deletes copy construction and copy assignment and designs move separately. Even a non-owning tail must refer to the live last node, so deleting the tail or clearing the list must update it to avoid a dangling pointer.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft forward_list, list and unique_ptr.
Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study